

Analysis of SDLC Models Applied to a College Management System

Aim & Objectives

Aim: To analyze the Software Development Life Cycle (SDLC) models used in the development of a real-world software application and identify the most suitable SDLC model.

Objectives:

- Study different SDLC models and their core operational mechanics.
- Analyze the specific workflow and development requirements of a College Management System.
- Compare the advantages, limitations, and key evaluation criteria of various SDLC models.
- Recommend and justify the most appropriate SDLC methodology for the application.

Selected Software Application

Application: College Management System (e.g., Campus ERP / Academic Portal)

A College Management System is an enterprise platform designed to automate multi-departmental institutional workflows. Key capabilities include:

- Managing student admissions, enrollments, and academic profiles.
- Publishing course schedules, class timetables, and academic syllabi.
- Tracking daily attendance and generating institutional compliance reports.
- Processing tuition/hostel fee collection, fine structures, and payment receipts.
- Conducting online examinations, evaluation, and automated CGPA/GPA scorecard generation.
- Managing central library inventory, issue records, and campus notification bulletins.

Functional Requirements by Module

Student Module

- User Registration & Student Profile
- Course Schedule & Timetable Access
- Attendance Tracking & Percentage Reports
- Assignment Submission & Homework Access
- Internal Marks & CGPA/GPA Results
- Online Tuition & Fee Payment Gateway

Examination & Grading Module

- Exam Scheduling & Automated Hall Tickets
- Question Paper Repository & Secure Uploads
- Score Entry & Automated Credit Engines
- Digital Marksheet Generation & Verification

Faculty Module

- Daily Attendance Entry & Reports
- Upload Study Materials & Assignments
- Grade Entry for Internal Assessment & Lab Exams
- Teaching Schedule & Duty Roster View
- Faculty Leave Application & Substitution Tracking

Admin & Finance Module

- Admission Processing & Document Auditing
- Staff Payroll & Human Resource Management
- Fee Configuration & Automated Fine Tracking
- Library Circulation & Transport Allocation
- Executive Dashboards & Institutional Analytics

SDLC Models Analysis

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Simple, linear structure; easy to manage with defined phases.	Inflexible; difficult to accommodate changing academic rules or mid-term requirements.
Prototype	Medium	Enhances early user understanding and refines visual UI design.	Increases overall development cost and risks project timeline creep.

SDLC Model	Suitability	Advantages	Limitations
Spiral	High	Strong risk management and systematic evaluation phases.	High cost and complexity for routine feature updates and module additions.
Agile	Very High	Continuous improvement; frequent feedback loops from faculty and admin.	Requires continuous active engagement from stakeholders and tech teams.
DevOps	Very High	Continuous Integration & Deployment (CI/CD) for rapid updates.	Demands robust automated testing pipelines and cloud infrastructure expertise.

Why Agile is Best for College Management Systems

Key Justifications:

- **Dynamic Regulations:** Academic policies, grading schemes, and regulatory standards change frequently.
- **Modular Deployment:** Key modules (Admissions, Exams, Fees) are built, tested, and shipped incrementally.
- **Rapid Bug Resolution:** Critical post-launch issues (payment failures, exam portal glitches) can be patched immediately.
- **Responsive UI/UX:** Portal interfaces require continuous refinement based on student and faculty feedback.

Agile Development Process Workflow



Detailed SDLC Phase Execution

1. Requirement Analysis

Gathering academic policies, grading formulas, and alert rules. Deliverable: **SRS Document**.

2. System Design

Designing ER diagrams, REST APIs, and UI wireframes. Deliverables: **UML Diagrams & DB Schema**.

3. Development

Coding core modules across modern tech stacks:

- **Backend:** Java (Spring Boot) / Python / Node.js
- **Frontend:** React / Angular / Vue.js
- **Database:** PostgreSQL / MySQL / MongoDB

4. Testing

Performing Unit, Integration, System, and User Acceptance Testing (UAT) with faculty/admins.

5. Deployment

Deploying application services across Cloud Infrastructure (AWS/Azure/GCP) or institutional servers alongside mobile releases.

6. Maintenance

Ongoing security patching, annual curriculum update configurations, monitoring, and backups.

Sample Verification Test Cases

Test ID	Action	Expected Result
TC_AUTH_01	Student Login with valid credentials	Successful authentication & student dashboard displayed
TC_AUTH_02	Login with incorrect password	Authentication fails; display "Invalid Credentials" error
TC_SUB_01	Submit assignment file (.pdf)	File uploaded to cloud storage; timestamped receipt shown
TC_PAY_01	Process semester fee payment	Transaction processed via gateway; official PDF receipt issued

Comparative Evaluation of SDLC Models

Evaluation Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Requirement Changes	Poor	Good	Excellent	Excellent	Excellent
Stakeholder Feedback	Low	High	High	Very High	Very High
Risk Handling	Low	Medium	High	Medium	High
Delivery Speed	Slow	Medium	Medium	Fast	Very Fast
Development Cost	Low	Medium	High	Medium	Medium
Suitability Score	No	Partially	Yes	Best	Best

Results & Conclusion

Results Summary: The College Management System is a complex, multi-user enterprise platform operating in a dynamic environment subject to shifting institutional policies. The Agile SDLC model provides the required flexibility through iterative sprints and continuous feedback. When paired with DevOps automated CI/CD pipelines, system stability and seamless deployments during peak traffic windows (e.g., exam results or admission days) are fully guaranteed.

Conclusion: Selecting the correct SDLC model depends heavily on application dynamics. For modern academic management systems, **Agile supported by DevOps practices** provides the optimal balance of adaptability, speed, system quality, and user satisfaction.